# Where Should Resident Layers Go? Optimal Placement for Weight Streaming in Offloaded LLM Inference

Quang-Vinh Dang[a,*], Dat Le[b], Minh Ngoc Dinh[c]

[a] *British University Vietnam, Hung Yen, Vietnam*
[b] *Department of Science, Technology, and International Projects, Ho Chi Minh City University of Economics and Finance, Ho Chi Minh City 700000, Vietnam*
[c] *Millennia Education, Ho Chi Minh City, Vietnam*

**Abstract**

Large language models that exceed GPU memory can be served by keeping some layers resident and streaming the others from host-attached memory, such as page-locked DRAM or a CXL memory expander, through a small ring of GPU buffers. We ask where the resident layers should be placed. We model the streaming pipeline as a max-plus linear system and show that its iteration time is the maximum mean of a family of tilings of the layer sequence, an exact evaluator for any placement. For layers of equal cost we prove three results: resident layers are useless when a layer computes at least as long as it takes to copy; otherwise, spreading the resident layers evenly attains the lower bound $\max(Lc, mf)$ whenever $(k-1)f \geq 2c$, so three buffers always suffice; and a contiguous block of resident layers, which is what filling GPU memory in model order produces, takes $\max(Lc,\ mf,\ rc+2c+(m-k)f)$, up to twice the optimum. We test the model on a Tesla T4 whose page-locked copies run at 12.3 GB/s, the bandwidth class of current CXL expanders, with real Llama-3.2-3B and Llama-3.1-8B layer kernels. Over 605 configurations the model predicts the measured iteration time with a mean error of 0.29% and orders placements correctly in 99% of distinguishable pairs. Even spreading is up to 1.50× faster than the contiguous prefix and needs two buffers where the contiguous placement needs six, and keeping the output head resident is 31.1% faster than streaming it. On the power-limited T4, clock changes

*Corresponding author. ORCID: 0000-0002-3877-8024
*Email addresses:* vinh.dq4@buv.edu.vn (Quang-Vinh Dang), datla@uef.edu.vn (Dat Le), minh.dinh@maeducation.com (Minh Ngoc Dinh)

caused by concurrent copies must be controlled; we show how they alter the measurements.

*Keywords:* LLM inference, weight offloading, prefetching, max-plus algebra, layer placement, host-attached memory

---

## 1. Introduction

A large language model whose weights do not fit in GPU memory can still be served on one GPU by keeping part of the weights in host-attached memory and copying the remaining layers into GPU memory while earlier layers compute [1–3]. The host-attached tier is usually page-locked DRAM; Compute Express Link (CXL) memory expanders extend it with capacity that the host maps as ordinary memory, and that a GPU reaches through the same host-initiated DMA path [4, 5]. On this path the transfer bandwidth is one to two orders of magnitude below GPU memory bandwidth (about 12 GB/s for PCIe Gen3 ×16, 10–20 GB/s for published CXL prototypes), so the copies are the bottleneck unless they overlap with computation.

Every such runtime makes two decisions that are rarely discussed separately: how many GPU buffers (*slots*) to reserve for layers in transit, and *which* layers to keep permanently resident in the remaining memory. The second decision has an obvious part, how many layers fit, and a less obvious part, *where* in the layer sequence the resident layers sit. Popular tools fill the GPU in model order: the Hugging Face Accelerate `device_map` assigns modules to the GPU until it is full and offloads the rest [3], which makes the resident layers a contiguous prefix of the model. FlexGen splits every layer between GPU and host by a fixed fraction [1], and an edge-serving system spreads retained layers evenly over its pipeline stages [6]. None of these choices is justified by an analysis of the transfer pipeline, and we did not find a study that quantifies how much the position of the resident layers matters.

This paper gives that analysis and checks it on a GPU. We model one inference iteration as a cyclic sequence of layer units, a single copy engine and a ring of $k$ slots, and we show that the steady-state iteration time is the maximum circuit mean of a max-plus linear system, which we characterise exactly. The model yields closed forms and a small set of rules:

1. **Exact iteration time** (Section 3). For any per-layer compute and transfer times, any resident set and any number of slots, the iteration

time is the maximum mean weight of a family of *tilings* of the layer sequence. It can be evaluated in time linear in the number of layers per iteration simulated, and it reproduces the measured iteration time on a real GPU with a mean error of 0.29% (Section 6.1).

2. **Optimal placement for uniform layers** (Section 4). If a layer computes at least as long as it takes to copy, any placement with two slots runs at the speed of resident weights, and resident layers buy nothing. Otherwise, spreading the resident layers evenly over the model attains the lower bound $\max(Lc, mf)$ whenever $(k-1)f \geq 2c$, so three slots always suffice and two suffice when a copy takes at least twice as long as a layer's computation.
3. **The cost of contiguous placement** (Section 4). For a contiguous resident block we derive the exact iteration time $\max(Lc,\ mf,\ rc + 2c + (m-k)f)$. It can be up to twice the optimum, and the worst case is the balanced regime in which the resident layers' computation equals the streamed layers' copies.
4. **Measurements on a GPU** (Section 6). With real layer kernels of two model shapes and page-locked host memory on a Tesla T4, evenly spread resident layers are up to 1.50× faster than the contiguous prefix chosen by model-order filling and reach the bound with two slots where the contiguous prefix needs six, and the model orders placements correctly in 99% of the comparisons.

The PCIe Gen3 path of the T4 sustains about 12.3 GB/s of page-locked copies, the same bandwidth class as the CXL expanders measured by others [4, 5]; we use it as a real instance of a host-mediated path and make no claim about CXL device latency. Section 7 discusses what the theory implies for faster paths and for CXL memory.

## 2. Background and related work

*Weight offloading for inference.* FlexGen formulates the placement of weights, activations and the key–value cache over GPU, CPU and disk as a linear program over per-layer fractions and overlaps I/O with computation [1]. ZeRO-Inference streams partitioned weights from CPU or NVMe memory [2]. Hugging Face Accelerate maps modules to devices in model order and moves offloaded weights to the GPU when a module runs [3]; llama.cpp offloads a chosen number of layers to the GPU and computes the others on the CPU [7].

PowerInfer [8], Fiddler [9] and LIA [10] split computation between CPU and GPU instead of moving weights, and LLM in a flash reduces the bytes read from flash [11]. LOIP spreads retained and offloaded layers evenly over the stages of an edge pipeline to overlap loading with computation [6]. Our results explain why even spreading works, when it is optimal, and what contiguous placement costs, for the single-GPU pipeline that all of these systems contain.

*Host-attached and CXL memory.* CXL Type-3 expanders appear to the host as memory without a local processor [12, 13]. A GPU cannot address them directly; weights reach GPU memory by a host-initiated copy from a page-locked region, and TraCT reports that copies from an unpinned CXL region go through a bounce buffer [4]. ITME prefetches model weights layer by layer from a CXL-hybrid memory and reports about 18 GB/s [5]. DAK lets the GPU read remote memory directly [14], a different path from the one modelled here. For our model, a CXL expander and host DRAM differ only in the transfer time per layer.

*Swapping in training.* Training systems swap tensors along a known dataflow graph: vDNN [15], SwapAdvisor [16], AutoTM [17] and Capuchin [18]. They decide what to evict and when to prefetch over one pass of a graph; inference repeats the same layer sequence every iteration with read-only weights, which is what makes a steady-state analysis possible.

*Max-plus algebra.* Synchronised discrete-event systems such as pipelines with buffers are linear in the max-plus algebra, and their asymptotic cycle time is the maximum circuit mean of the associated event graph [19], computable for instance with Karp's algorithm [20]. We apply this framework to the weight-streaming pipeline and exploit the structure of its circuits to obtain closed forms.

## 3. Model

*Units and iterations.* An inference iteration (one prefill chunk or one decode step of a batch) executes $L$ *units* in a fixed order $0, 1, \ldots, L-1$; a unit is a transformer layer, or any other block whose weights move as one piece, such as the output head. Iterations repeat. Unit $i$ takes $c_i > 0$ to compute once its weights are in GPU memory and $f_i > 0$ to copy from host-attached memory to the GPU; $c_i$ depends on the number of tokens in the iteration and $f_i = a + s_i/B$ on the unit's size $s_i$, the path bandwidth $B$ and a per-copy cost $a$.

*Resident set and slots.* A set $R$ of $r$ units stays in GPU memory. The other $m = L - r$ units, $s_0 < s_1 < \cdots < s_{m-1}$, are *streamed* every iteration through $k$ GPU slots used as a ring, $1 \le k \le m$. We number streamed transfers globally: transfer $j \ge 0$ carries unit $\sigma(j) = s_{j \bmod m}$ of iteration $\lfloor j/m \rfloor$ into slot $j \bmod k$.

*Assumptions.* (A1) One copy engine performs the transfers one at a time in the order $j = 0, 1, \ldots$. (A2) The GPU computes units one at a time in program order. (A3) Transfer $j$ may start as soon as the previous transfer has finished and its slot is free, that is, once the computation of the unit carried by transfer $j - k$ has finished (work-conserving prefetching). (A4) Times are deterministic and the same in every iteration. A1–A3 describe a runtime with one copy stream, one compute stream and events between them, which is how the runtime of Section 5 is built; A4 is relaxed in the measurements.

*Recurrences.* Let $X_j$ be the completion time of transfer $j$ and $Y_n$ the completion time of the $n$-th unit execution (unit $n \bmod L$ of iteration $\lfloor n/L \rfloor$), with $\nu(j)$ the execution index of transfer $j$'s unit. Then

$$X_j = \max\big(X_{j-1},\, Y_{\nu(j-k)}\big) + f_{\sigma(j)}, \tag{1}$$

$$Y_n = \max\big(Y_{n-1},\, X_j\big) + c_{n \bmod L}, \tag{2}$$

where in (2) $j$ is the transfer used by execution $n$; for a resident unit the second argument of the maximum is absent and $Y_n = Y_{n-1} + c_{n \bmod L}$. Terms of negative index are zero. These are max-plus linear recurrences: only maxima and additions occur.

*Event graph and tilings.* Draw one node per event $X_j$ and $Y_n$ and an arc from every event on the right-hand side of (1)–(2) to the event on the left, weighted with the time added ($f$ for arcs into $X$, $c$ for arcs into $Y$). Shifting all indices by one iteration ($m$ transfers, $L$ executions) maps the graph onto itself, so the system is periodic. A walk that starts at a node and returns to the same node shifted by $w \ge 1$ iterations is a *circuit spanning $w$ iterations*; its weight is the sum of its arc weights.

**Lemma 1** (Iteration time)**.** *Under A1–A4 the iteration time $P(R, k) = \lim_{n\to\infty} Y_{nL}/n$ exists and equals the maximum, over all circuits, of the circuit weight divided by the number of iterations it spans.*

*Proof.* Collecting the events of one iteration into a vector $z(n)$ gives $z(n) = A_0 \otimes z(n) \oplus A_1 \otimes z(n-1)$ in max-plus notation, with $A_0$ acyclic because arcs inside an iteration point forward in time. Hence $z(n) = A_0^* A_1 \otimes z(n-1)$, and the cycle time of this recurrence is the maximum circuit mean of the graph of $A_0^* A_1$ [19, Ch. 3]. Circuits of that graph correspond to circuits of the event graph, with the number of iterations spanned as the length. □

A circuit can only move forward: from $Y_n$ to $Y_{n+1}$ (weight $c$), from the execution of transfer $j$'s unit to transfer $j+k$ (the slot arc, weight $f$), from $X_j$ to $X_{j+1}$ (weight $f$) and from $X_j$ to the execution that uses it (weight $c$). Reading a circuit in time order therefore gives the following description, which we use in all proofs.

**Lemma 2** (Tilings)**.** *Every circuit spanning $w$ iterations either uses only transfer nodes, in which case its weight is $w \sum_{i \notin R} f_i$, or it partitions the $wL$ unit executions of $w$ consecutive iterations into* singles*, each one execution $n$ with weight $c_{n \bmod L}$, and* segments*. A segment begins right after the execution of a streamed unit carried by transfer $j$, ends with the execution of the unit carried by transfer $j+k+t$ for some $t \geq 0$, contains every execution in between, and has weight $\sum_{q=j+k}^{j+k+t} f_{\sigma(q)} + c_{\sigma(j+k+t)}$. Conversely, every such partition is a circuit.*

A segment contains $k+t$ streamed executions and all resident executions between them; compared with covering the same executions by singles it changes the weight by its *gain*. For uniform units ($c_i = c$, $f_i = f$) and a segment that contains $q = k+t$ streamed units and $G$ resident units,

$$\text{gain} = (t+1)f + c - c\,(q+G) = (q-k+1)(f-c) - (k-2)\,c - c\,G. \quad (3)$$

The iteration time is therefore $Lc$ plus the largest average gain per iteration over all tilings, or the transfer-only bound, whichever is larger.

**Proposition 1** (Lower bound)**.** *For every resident set and every $k$, $P(R,k) \geq \max\bigl(\sum_i c_i,\ \sum_{i \notin R} f_i\bigr)$.*

*Proof.* The tiling made only of singles and the transfer-only circuit are circuits spanning one iteration. □

## 4. Placement theory

Throughout this section units are uniform: $c_i = c$ and $f_i = f$. This is the case of a dense transformer whose units are its layers; the output head and other irregular units are treated numerically in Section 4.1. We write $m = L - r$ for the number of streamed units and assume $2 \le k \le m$.

*Gaps and balanced placements.* For a resident set $R$ let $g_j$ be the number of resident units between streamed units $s_j$ and $s_{j+1}$ (indices modulo $m$, wrapping around the iteration), so that $\sum_j g_j = r$.

**Definition 1.** A placement is *balanced* if any two windows of the same length of the cyclic gap sequence $(g_0, \dots, g_{m-1})$ have sums that differ by at most one.

In a balanced placement every window of $q$ consecutive gaps sums to $\lfloor qr/m \rfloor$ or $\lceil qr/m \rceil$, since the average over all windows of length $q$ is $qr/m$; this extends to windows longer than $m$, which contain whole turns of the cycle. Balanced placements exist for every $r$ and $m$: the gaps $g_j = \lfloor (j+1)r/m \rfloor - \lfloor jr/m \rfloor$ form a balanced (Christoffel) word [21, Ch. 2]. In words, the resident layers are spread as evenly as the integers allow.

**Theorem 1** (Per-layer compute bound)**.** *If $f \le c$ and $k \ge 2$, then $P(R, k) = Lc$ for every resident set $R$.*

*Proof.* By (3), every segment has gain $(q - k + 1)(f - c) - (k - 2)c - cG \le 0$. Hence every tiling has weight at most $wLc$, and the transfer-only circuit has weight $wmf \le wLc$. Proposition 1 gives the reverse inequality. □

Theorem 1 says that resident layers are useless when a layer's computation hides its own copy: two slots (double buffering) already run at the speed of fully resident weights. The interesting case is $f > c$, in which copies are slower than computation per layer and resident layers must supply the missing computation time.

**Theorem 2** (Balanced placement is optimal)**.** *If $f > c$, $k \ge 2$, $(k-1)f \ge 2c$ and the placement is balanced, then $P(R, k) = \max(Lc, mf)$, the lower bound of Proposition 1. In particular three slots always suffice, and two suffice when $f \ge 2c$.*

*Proof.* Consider a tiling spanning $w$ iterations with segments $1, \dots, N$, where segment $\ell$ contains $q_\ell$ streamed and $G_\ell$ resident units. The resident units of segment $\ell$ are those of $q_\ell$ consecutive gaps, so $G_\ell \geq q_\ell r/m - 1$ by balance. By (3),

$$\begin{aligned}\text{gain}_\ell &\leq (q_\ell - k + 1)(f - c) - (k-2)c - c\,(q_\ell r/m - 1) \\ &= q_\ell \,\frac{mf - Lc}{m} \;-\; \big((k-1)f - 2c\big),\end{aligned}$$

using $L = m + r$. The second term is non-positive by assumption. Segments are disjoint, so $\sum_\ell q_\ell \leq wm$. If $mf \geq Lc$, the weight of the tiling is at most $wLc + \sum_\ell q_\ell(mf - Lc)/m \leq wLc + w(mf - Lc) = wmf$. If $mf < Lc$, every gain is negative and the weight is at most $wLc$. The transfer-only circuit has weight $wmf$. By Lemma 1, $P \leq \max(Lc, mf)$, and Proposition 1 gives equality. □

**Theorem 3** (Contiguous placement)**.** *If $f > c$, $k \geq 2$ and the resident units form one contiguous block, then*

$$P(R,k) = \max\big(Lc,\ mf,\ rc + 2c + (m-k)f\big).$$

*Proof. Lower bound.* Besides the two circuits of Proposition 1, take the one-iteration tiling that covers the resident block and the first streamed unit after it by singles and the remaining $m - 1$ streamed units by one segment with $t = m - 1 - k$; its weight is $rc + c + (m-k)f + c$. (If $m = k$ this term is at most $Lc$.)

*Upper bound.* Take a tiling spanning $w$ iterations. Call an occurrence of the resident block *uncovered* if no segment contains it. A segment that contains the first streamed unit after an occurrence of the block also contains the block, so after an uncovered occurrence that unit is a single. The uncovered occurrences split the $w$ iterations into stretches; consider a stretch of $v$ iterations that contains $N \geq 1$ segments (a stretch without segments has weight $vLc$). It has $v - 1$ covered occurrences of the block and at most $vm - 1$ streamed units in segments. Summing (3) over its segments, and using $(k-1)(f-c) + (k-2)c = (k-1)f - c > 0$,

$$\begin{aligned}\text{weight} &\leq vLc + (f-c)(vm-1) - N\big((k-1)f - c\big) - c\,r\,(v-1) \\ &\leq vLc + (f-c)(vm-1) - \big((k-1)f - c\big) - c\,r\,(v-1) \\ &= v\,mf + \big(rc + 2c - kf\big),\end{aligned}$$

so its weight per iteration is at most $mf + (rc + 2c - kf)/v \le \max\big(mf,\ rc + 2c + (m-k)f\big)$. If no occurrence is uncovered, all $w$ block occurrences lie in segments, $\sum_\ell q_\ell \le wm$, and the same computation without the $-1$ terms gives at most $w\,mf$. Every circuit thus has mean at most $\max(Lc, mf, rc + 2c + (m-k)f)$. □

The third term has a direct reading. While the resident block computes, the copy engine can load only the $k$ streamed units that fit in the slots and then waits; the remaining $m - k$ copies happen while streamed units run at the copy rate, so the resident computation is not hidden behind copies.

**Corollary 1** (Cost of contiguous placement)**.** *Under the assumptions of Theorems 2 and 3, the contiguous placement is slower than the balanced one by the factor*

$$\frac{\max(Lc,\ mf,\ rc + 2c + (m-k)f)}{\max(Lc,\ mf)} < 2.$$

*The factor approaches 2 when $rc \approx mf$ and $kf$ and $c$ are small compared with $rc$.*

*Proof.* Because $kf > 2c$, $rc + 2c + (m-k)f < rc + mf \le Lc + mf \le 2\max(Lc, mf)$. For the limit take $r = m$, $f = c(1+\epsilon)$ and $m \to \infty$ with $k$ fixed. □

The penalty therefore peaks exactly where streaming is most attractive: when the resident layers' computation is about as long as the streamed layers' copies, a balanced placement hides all copies behind computation and a contiguous one exposes almost all of them.

**Proposition 2** (Two slots in the band $c < f < 2c$)**.** *If $c < f < 2c$, $k = 2$ and the placement is balanced, then* $\max(Lc, mf) \le P(R, 2) \le \max(Lc, mf) + \frac{m}{2}(2c - f)$.

*Proof.* In the proof of Theorem 2 the second term of each gain is $2c - f > 0$, and every segment contains at least $k = 2$ streamed units, so there are at most $wm/2$ segments. □

In the band the bound is loose: over 3 000 random instances the excess was at most a third of it, and for $L \le 12$ a brute-force search over all placements found no placement better than the balanced one. Because a slot uses the same memory as a resident layer, two slots plus one more resident layer is often faster than three slots; the exact evaluator of Lemma 1 decides this in each case (Section 6.3).

*Guidelines.* For a dense model with layers of equal size the analysis gives a complete recipe. (i) If a layer's computation is at least its copy time, use two slots and no resident layers beyond what is free. (ii) Otherwise use two slots if $f \geq 2c$ and three if $c < f < 2c$ (or evaluate both), fill the remaining memory with resident layers, and spread them evenly. (iii) The resulting iteration time is $\max(Lc, mf)$, so the number of resident layers needed to reach compute speed is $r \geq L(1 - c/f)$.

### *4.1. Non-uniform units*

Real models are not perfectly uniform: the output head of Llama-3 holds as many parameters as several layers but computes for only the last token in generation. Lemma 1 still gives the exact iteration time, and the recurrences (1)–(2) evaluate it by simulating a few hundred iterations, in time proportional to the number of unit executions. We use this evaluator inside a local search that swaps, adds and removes resident units subject to the memory budget (resident bytes plus $k$ slots sized for the largest streamed unit) and compare it with the balanced placement of the layers alone (Section 6.4).

## 5. Measurement methodology

*Platform.* All measurements use a Tesla T4 (16 GB GDDR6, 40 streaming multiprocessors, PCIe Gen3 $\times$16) on a Google Colab virtual machine with 13.6 GB of host memory, PyTorch 2.14.1 and CUDA 13.0. Page-locked host-to-GPU copies of a layer (201 MB) run at 12.3 GB/s. The T4 has a 70 W power limit, and under sustained load its driver lowers the SM clock to stay within it. In a first set of runs this power capping changed the compute time of a layer by up to 19% within one configuration, and a concurrent copy, which itself draws power, slowed computation by 2–9%. Because such clock changes are a property of this power-limited card rather than of the transfer pipeline, the measurements reported in Section 6 lock the SM clock with `nvidia-smi -lgc` at a frequency at which the card stays below its power limit (570 MHz for iterations of up to 1 024 tokens and 450 MHz for longer ones). A sampler records the SM clock, power and throttle state every 200 ms, so that every configuration can be checked for clock changes. Section 6.5 reports what happens without the lock.

*Model and workloads.* We use the layer shapes of Llama-3.2-3B [22] (28 layers, model width 3 072, 24 query and 8 key–value heads of dimension

128, feed-forward width 8 192) with random FP16 weights; execution time does not depend on the weight values. A unit is one transformer layer: RMS normalisation, the fused query–key–value projection, attention, the output projection, a second normalisation and the gated feed-forward block. Rotary embeddings are omitted. Attention uses the memory-efficient kernel of PyTorch with the key–value heads expanded to the query heads, because the T4 does not support the fused grouped-query path. Prefill iterations process one prompt of $T \in \{256, 512, 768, 1024, 1536, 2048\}$ tokens with causal attention; decode iterations process one new token for each of $B \in \{16, 64\}$ sequences against a key–value cache of 512 tokens that stays in GPU memory. One cache tensor is shared by all layers, which does not change the work per layer and keeps GPU memory free for resident layers. With these shapes the ratio $f/c$ of copy to compute time per layer ranges from 0.46 to 5.65, which covers the three regimes of Section 4.

*Runtime.* The runtime implements assumptions A1–A3 directly. Every layer's weights are one contiguous slice of a single page-locked host buffer. Resident layers are copied to GPU memory once. Streamed layers use $k$ GPU slots of one layer each, a copy stream and a compute stream: copy $j$ waits on an event recorded after the computation of the layer that used slot $j \bmod k$ before it, and the computation of a streamed layer waits on the event recorded after its copy. All work is enqueued asynchronously, so the GPU, not the host, decides when each operation starts; the host enqueue time per iteration is recorded and was always shorter than the iteration. For each configuration the runtime executes two untimed iterations and then 12 timed prefill iterations (40 for decode), discards the first 4 (10), and reports the mean iteration time over the rest. One further iteration records the start and end of every computation and copy with CUDA events, which gives the timelines in Figure 3.

*Configurations.* For every Llama-3.2-3B workload we run eight resident counts ($r = 0, 4, 7, 10, 14, 18, 21, 24$ of 28 layers), two slot counts ($k = 2, 3$) and four placements: the first $r$ layers resident (*contiguous-first*, the order in which model-order filling assigns layers), the last $r$ layers (*contiguous-last*), the balanced placement of Section 4 (resident positions $\lfloor (j + \frac{1}{2})L/r \rfloor$), and one random placement. For the Llama-3.1-8B shapes (16 layers, to fit host memory) we use $r = 0, 4, 8, 12$ with prefill of 512 and 1 024 tokens and decode with $B = 16$; for the model with the output head as an extra unit, $r = 0, 7, 14, 21$ with prefill of 512 tokens and decode with $B = 16$. A slot

sweep at $T = 768$ and $r = 14$ uses $k = 1, 2, 3, 4, 6$. In total the measurements cover 605 streaming configurations plus one all-resident run per workload.

*Predictions.* Before the streaming runs of a workload we measure, with all weights resident, the compute time $c_i$ of every layer (median of 7 timed repetitions after warm-up), and once per model the copy time $f_i$ of every layer. The prediction for a configuration is the iteration time given by Lemma 1 for these measured $c_i$ and $f_i$; nothing is fitted to the streaming runs. We also measure every layer's compute time while copies run back to back on the copy stream ($c'_i$) and evaluate a variant of the model in which a layer computes at rate $1/c'_i$ while a copy is in progress and at rate $1/c_i$ otherwise. With locked clocks $c'_i$ and $c_i$ differ by less than 3.6%, so the two predictions coincide; the variant matters only for the unlocked runs.

## 6. Results

### *6.1. Accuracy of the model*

Figure 1 compares the predicted and measured iteration times of all 605 streaming configurations of the main runs (clocks locked, both model shapes, prefill and decode). The prediction uses the per-layer compute times scaled so that their sum equals the measured all-resident iteration (scale factors between 0.97 and 1.00), and the per-layer copy times; nothing is fitted to the streaming runs. The mean absolute error is 0.29% and the largest error 5.24%. Without the scaling the mean error is 0.39%. For the Llama-3.1-8B layer shapes the mean error is 0.48%. The few errors above 2% have identifiable causes. The largest, 5.24%, occurs for the 8B shapes at $T = 512$, where the heavier layers drew enough power that the clock fell to 433–445 MHz despite the 450 MHz lock. The others occur in decode iterations with most layers resident, which last less than about 120 ms and in which the host takes about as long to enqueue the work as the GPU takes to run it; the model contains no host overhead. Two runs of the same 46 configurations on different Colab machines differed by 1.74% on average (at most 3.05%), so the model is more precise than the variation between machines; this is why the compute times are measured on the machine whose iterations are predicted. Over the 285 pairs of placements with the same workload, resident count and slots whose measured times differ by more than 2%, the model orders 99% correctly.

*6.2. Placement*

Figure 2 shows, for each prefill length, the iteration time of the contiguous and balanced placements relative to the lower bound $\max(\sum c_i, \sum_{\text{streamed}} f_i)$ as the number of resident layers grows. The three regimes of Section 4 are visible. In the transfer-bound workloads ($T \leq 768$, $f/c$ between 1.28 and 4.16) the balanced placement stays at the bound for every $r$, while the contiguous one leaves the bound as soon as resident computation exceeds what $k$ slots can prefetch, and the gap widens until the resident layers' computation matches the streamed layers' copies. The largest measured gap is 1.50× (prefill of 256 tokens, $r = 21$, $k = 2$), and Theorem 3 predicts 1.50×. Contiguous-last behaves like contiguous-first, as the theory expects for any single block. Random placements usually come close to balanced ones but not always: the worst random placement (decode with 64 sequences, $r = 14$, $k = 2$) was 14% slower than the balanced one. When a layer computes at least as long as it copies ($T \geq 1024$; at $T = 1\,024$ the two times are equal), all placements, including streaming every layer, are within 0.1% of each other and of the all-resident time, as Theorem 1 states. In decode the copies dominate unless almost all layers are resident, and placement matters only in the few configurations in which $rc$ approaches $mf$.

Figure 3 shows why. In the contiguous configuration the copy engine fills the slots while the resident block computes and then waits for most of the block; afterwards the streamed layers run at the copy rate. In the balanced configuration the copy engine is never idle and the iteration ends when the last copy has been consumed.

*6.3. Number of slots*

At $T = 768$ a layer copies in 1.28 times its compute time, inside the band $c < f < 2c$ of Proposition 2. Figure 4 shows the slot sweep for $r = 14$. One slot serialises copy and computation for both placements. The balanced placement reaches the lower bound with two slots already: the excess that Proposition 2 allows in this band does not occur for these 14 resident layers. The contiguous placement improves with every additional slot, because each slot lets one more copy proceed during the resident block, and needs six slots to reach the bound, four more than the balanced placement. Since a slot occupies as much memory as a resident layer, the balanced placement reaches the same speed with four layers' worth of memory to spare. The model reproduces the sweep with a mean error of 0.16%.

*6.4. Non-uniform units: the output head*

With the Llama-3.2 output head as an extra unit (788 MB, four times a layer, but computed for one token only), the model predicts the measured iterations with a mean error of 0.41%. Figure 5 uses the measured per-unit times to compare placements under a memory budget. Streaming the head forces every slot to the head's size, so keeping the head resident and spreading the layers over the rest of the budget is 31.1% faster than the balanced layer placement with a streamed head at the median budget; a local search over all placements improves on this rule by only 0.1%.

*6.5. Without clock locking*

In runs whose SM clock fell below the locked value because of the power cap, the model with compute times from a cooler phase errs by up to 19.13% (mean 8.43%). Scaling each configuration's compute times by the ratio of the clock during the compute measurement to the clock during the run removes almost all of it (mean 0.63%, maximum 2.92%), so the error is the clock, not the pipeline model. In our first, fully unlocked runs the effect changed a conclusion: for a 1 024-token prefill with 14 resident layers the balanced placement measured 374 ms against 364 ms for the contiguous one, although the model predicts the opposite. These runs had no clock samples, so we can only point to the mechanism shown above: with a balanced placement copies overlap computation for the whole iteration, which raises power draw and lowers the clock exactly while the computation is on the critical path.

## 7. Discussion

*What runtimes should do.* The rules of Section 4 cost nothing to apply. A runtime that streams layers with prefetching should spread its resident layers evenly instead of filling GPU memory in model order, and should size its ring by the ratio of copy to compute time per layer: two slots when a copy takes at least twice as long as a layer's computation, three otherwise, or the faster of the two according to the evaluator, which needs only per-layer compute and copy times that a runtime can measure in one iteration. When a layer computes longer than it takes to copy, resident layers do not shorten the iteration at all, and the memory is better spent on a larger batch or key–value cache. When copies dominate even with every free byte resident, placement does not matter either, and only fewer bytes (quantisation) or a faster path help.

*Faster paths and CXL memory.* The analysis depends on the path only through the copy time per layer $f$. A CXL Type-3 expander reached through a page-locked host mapping is such a path [4, 5]; with the 10–20 GB/s reported for current prototypes, $f$ is in the range we measured on PCIe Gen3, and the regimes, the optimality of balanced placement and the cost of contiguous placement carry over unchanged. A faster path, such as several expanders interleaved or PCIe Gen5, lowers $f$ and moves a given workload towards the regime $f \leq c$ in which placement no longer matters; a GPU that computes faster moves it the other way. We did not measure a CXL device, and device-specific effects that our model does not contain, such as latency tails under contention or interaction with host memory traffic, would enter through per-transfer times that are no longer constant.

*Power-limited GPUs.* On the power-limited T4 a concurrent copy and long runs lowered the SM clock, which changes $c$ during an iteration and breaks assumption A4. The effect was large enough to reverse the order of two placements in one configuration of our first runs (Section 6.5). The contention variant of the model captures the part caused by concurrent copies but not the slower drift of the clock with temperature and power. Data-centre GPUs with higher power limits are less affected, but any deployment that runs close to its power limit should measure $c$ under the same conditions as the iteration it wants to predict.

*Limitations.* (i) All measurements are on one GPU type with one model family; we checked a second model shape (Llama-3.1-8B layers) but not other GPUs, and the clock lock that makes the measurements reproducible also makes them slower than default operation. (ii) We stream whole layers; a runtime that splits layers into tensors has more, smaller units, to which Lemma 1 applies unchanged but for which we did not measure. (iii) The key–value cache stays in GPU memory; streaming it as well adds per-layer copy time that grows with the context, which fits the model as a larger $f_i$ but was not measured. (iv) We assume one copy engine and in-order execution; runtimes that reorder layers or split copies across several engines need a different event graph. (v) The theorems cover layers of equal cost; for non-uniform units we offer an exact evaluator and a search, not a closed form.

## 8. Conclusion

Streaming the layers of a model through a small GPU-memory ring is a max-plus linear system, and its iteration time is the maximum mean of a family of tilings of the layer sequence. For layers of equal cost this gives a complete answer to the placement question. When a layer computes at least as long as its copy takes, two slots suffice and resident layers are useless. Otherwise, spreading the resident layers evenly attains the lower bound $\max(Lc, mf)$ as soon as $(k-1)f \geq 2c$, while a contiguous block, the result of filling GPU memory in model order, takes $\max(Lc, mf, rc+2c+(m-k)f)$, up to twice as long. On a Tesla T4 with page-locked host memory, the same bandwidth class as current CXL expanders, the model predicted 605 measured iteration times with a mean error of 0.29%, and even spreading was up to 1.50× faster than the contiguous prefix. The rules are cheap enough for any offloading runtime to adopt; the open questions are tensor-level units, streamed key–value caches and power-limited operation, in which the per-layer times themselves depend on the schedule.

### CRediT authorship contribution statement

**Quang-Vinh Dang**: Conceptualization, Methodology, Formal analysis, Software, Investigation, Writing – original draft, Visualization, Supervision. **Dat Le**: Validation, Writing – review & editing. **Minh Ngoc Dinh**: Validation, Writing – review & editing.

### Data availability

The model code, the GPU runtime, the raw measurements and the scripts that produce every number, table and figure are available at `https://github.com/vinhqdang/Enabling-CXL-Based-Memory-Disaggregation-for-Large-Scale-AI-Systems`.

### Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used generative AI-assisted tools for software implementation, experiment scripting and drafting and editing of text. After using these tools, the authors reviewed, verified and edited all code, results and content as needed and take full responsibility for the content of the published article.

## References


[1] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, C. Zhang, FlexGen: High-throughput generative inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML '23), 2023. URL https://proceedings.mlr.press/v202/sheng23a.html

[2] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, Y. He, ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2021.

[3] Hugging Face, Accelerate documentation: Loading big models into memory, https://huggingface.co/docs/accelerate/en/concept_guides/big_model_inference, accessed 8 November 2026 (2026).

[4] D. Yoon, Y. Min, H. Kim, S. H. Noh, J. Kim, TraCT: Disaggregated LLM serving with CXL shared memory KV cache at rack-scale (2025). arXiv:2512.18194.

[5] H. Jang, Y. Min, S. Kim, T. Ahn, H. Kim, Y. Joo, H. Kim, J. Kim, ITME: Inference tiered memory expansion with disaggregated CXL-hybrid memories (2026). arXiv:2606.12556.

[6] M. Sun, X. Zhang, S. Qu, Y. Li, M. Xiao, Y. Zheng, Y. Yuan, D. Yu, LOIP: Collaborative lossless LLM inference serving with offloading-based pipeline parallelism on edge devices (2025). arXiv:2512.21835.

[7] G. Gerganov, contributors, llama.cpp: LLM inference in C/C++, https://github.com/ggml-org/llama.cpp (2023).

[8] Y. Song, Z. Mi, H. Xie, H. Chen, PowerInfer: Fast large language model serving with a consumer-grade GPU, in: Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24), ACM, New York, NY, USA, 2024, pp. 590–606. doi:10.1145/3694715.3695964.
URL https://arxiv.org/abs/2312.12456

[9] K. Kamahori, T. Tang, Y. Gu, K. Zhu, B. Kasikci, Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models, in: Proc. Int. Conf. on Learning Representations (ICLR), 2025.

[10] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, N. S. Kim, LIA: A single-GPU LLM inference acceleration with cooperative AMX-enabled CPU–GPU computation and CXL offloading, in: Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA), 2025, pp. 544–558. doi:10.1145/3695053.3731092.

[11] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, M. Farajtabar, LLM in a flash: Efficient large language model inference with limited memory, in: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 12562–12584.

[12] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, R. Bianchini, Pond: Cxl-based memory pooling systems for cloud platforms, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 574–587. doi:10.1145/3575693.3578835.
URL https://doi.org/10.1145/3575693.3578835

[13] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, P. Chauhan, Tpp:

Transparent page placement for cxl-enabled tiered-memory, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 742–755. doi:10.1145/3582016.3582063.
URL https://doi.org/10.1145/3582016.3582063

[14] S. Lin, Z. Guo, J. Lin, DAK: Direct-access-enabled GPU memory offloading with optimal efficiency for LLM inference (2026). arXiv:2604.26074.

[15] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, S. W. Keckler, vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design, in: Proc. 49th IEEE/ACM Int. Symp. on Microarchitecture (MICRO), 2016, pp. 1–13.

[16] C.-C. Huang, G. Jin, J. Li, SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 1341–1355.

[17] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, V. Akella, AutoTM: Automatic tensor movement in heterogeneous memory systems using integer linear programming, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 875–890.

[18] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, X. Qian, Capuchin: Tensor-based GPU memory management for deep learning, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 891–905.

[19] F. Baccelli, G. Cohen, G. J. Olsder, J.-P. Quadrat, Synchronization and Linearity: An Algebra for Discrete Event Systems, Wiley, 1992.

[20] R. M. Karp, A characterization of the minimum cycle mean in a digraph, Discrete Mathematics 23 (3) (1978) 309–311.

[21] M. Lothaire, Algebraic Combinatorics on Words, Cambridge University Press, 2002.

[22] A. Dubey, et al., The llama 3 herd of models (2024). arXiv:2407.21783.

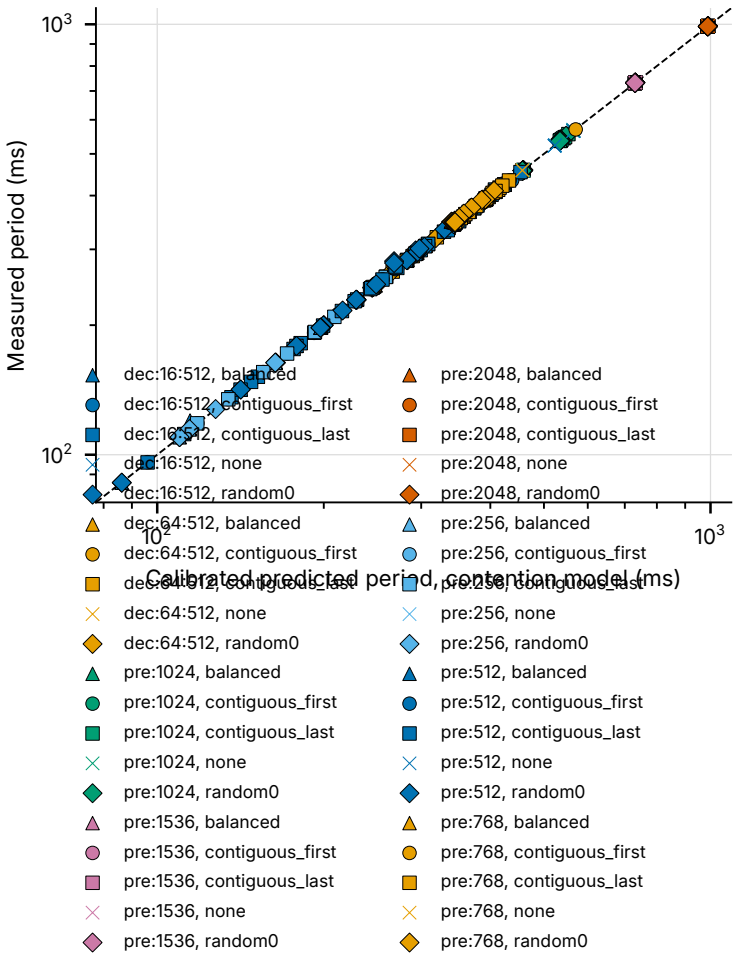



Figure 1: Predicted versus measured iteration time for all streaming configurations of the main runs (Tesla T4, locked clocks). The line is $y = x$.

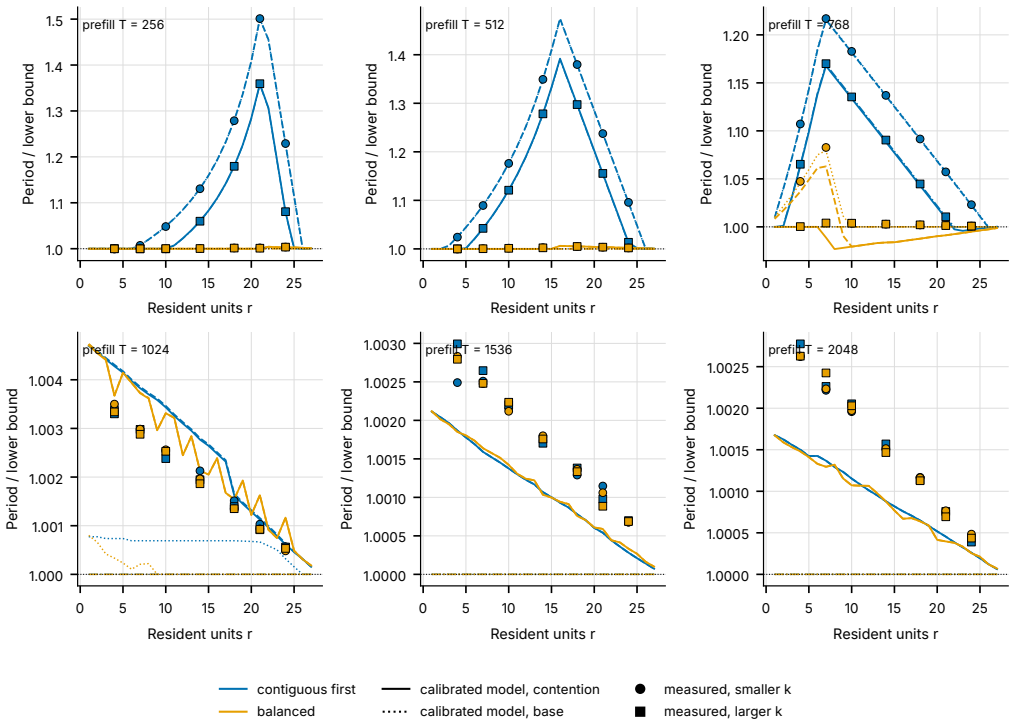



Figure 2: Iteration time relative to the lower bound versus the number of resident layers, for contiguous-first and balanced placements (markers: measured; lines: model).

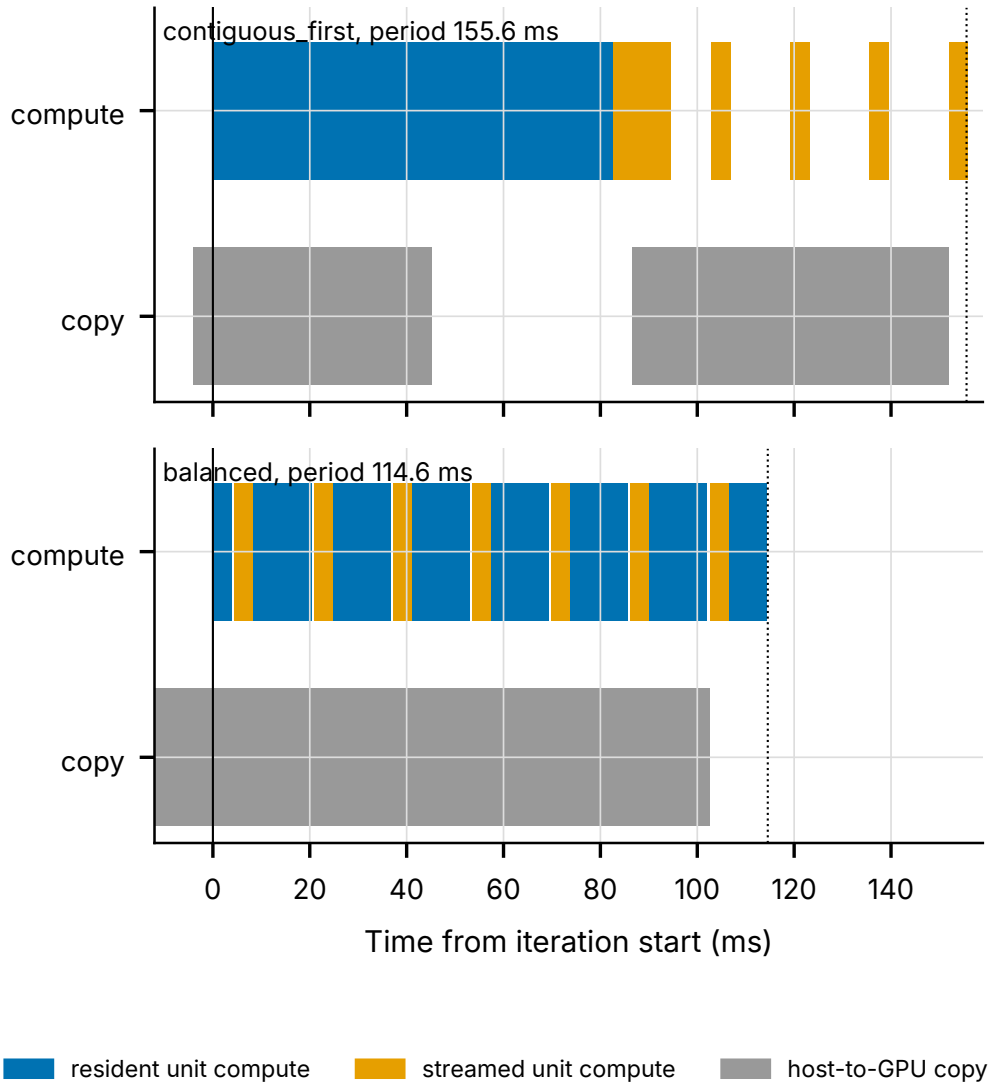



Figure 3: Measured timelines of one iteration (prefill of 256 tokens, $r = 21$, $k = 3$) for contiguous-first (top) and balanced (bottom) placement. Each panel shows computation (upper row) and copies (lower row).

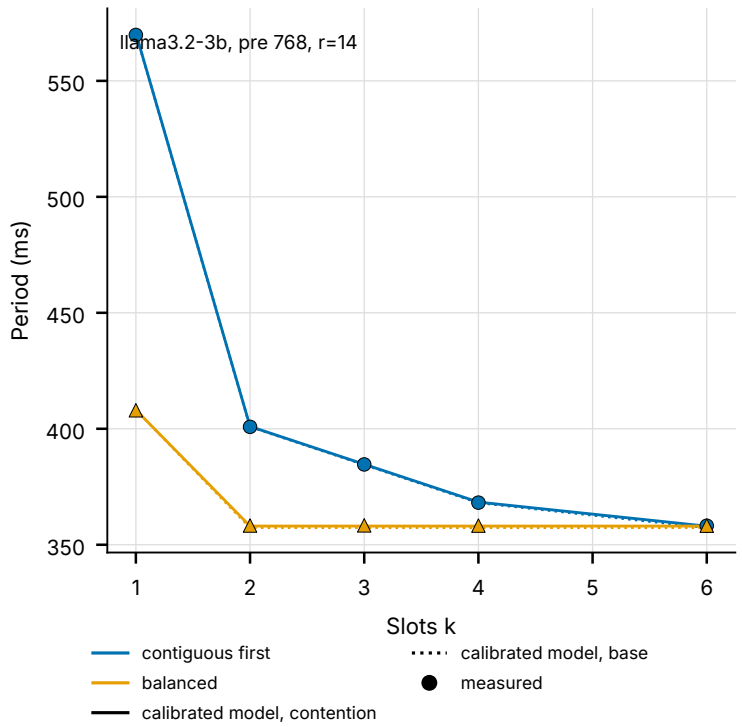



Figure 4: Iteration time versus the number of slots at $T = 768$, $r = 14$ (markers: measured; lines: model).

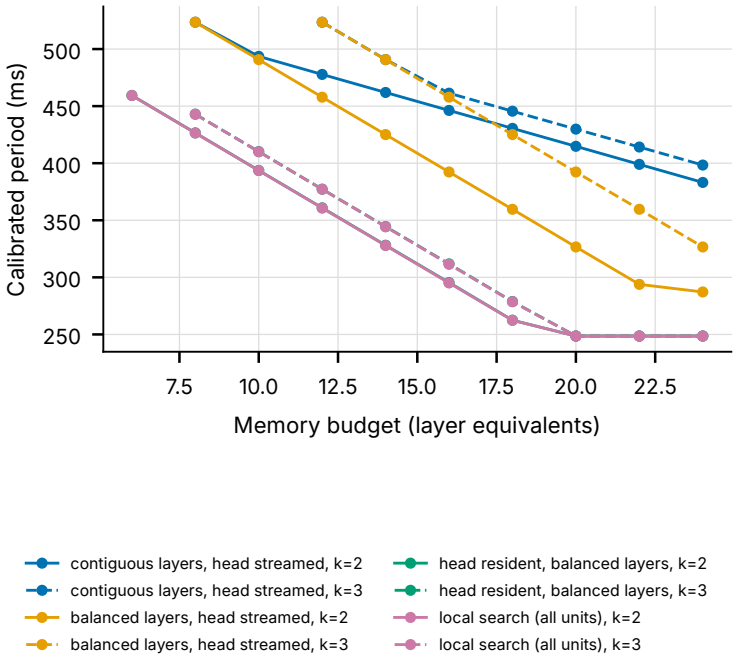



Figure 5: Predicted iteration time versus memory budget with the output head as a unit (prefill of 512 tokens): contiguous layers, balanced layers, head resident plus balanced layers, and local search.